

Amit_Lab5 - Day 5 Assignment

C Programming - Functions

Question 1: Menu-driven calculator using functions and switch-case

```
#include <stdio.h>
#include <math.h>
#define PI 3.14159265358979

float circle(float radius);
float rectangle(float length, float width);
float square(float side);
double power(double x, double y);
double squareRoot(double number);
void displaymenu(void);

int main()
{
    int choice;
    float radius, length, width, side;
    double base, num;

    do
    {
        displaymenu();

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter radius of the circle: ");
                scanf("%f", &radius);
                if (radius < 0)
                    printf("Error: radius cannot be negative.\n");
                else
                    printf("Area of circle = %.4f\n", circle(radius));
                break;

            case 2:
                printf("Enter length and width of the rectangle: ");
                scanf("%f %f", &length, &width);
                if (length < 0 || width < 0)
                    printf("Error: dimensions cannot be negative.\n");
                else
                    printf("Area of rectangle = %.4f\n", rectangle(length, width));
                break;

            case 3:
                printf("Enter side of the square: ");
                scanf("%f", &side);
                if (side < 0)
                    printf("Error: side cannot be negative.\n");
                else
                    printf("Area of square = %.4f\n", square(side));
                break;

            case 4:
                printf("Enter base and exponent (x y): ");
```

```

        scanf("%lf %lf", &base, &num);
        if (num < 0)
            printf("Error: exponent must be non-negative.\n");
        else
            printf("%.2f ^ %.2f = %.4f\n", base, num, power(base, num));
        break;

    case 5:
        printf("Enter a number: ");
        scanf("%lf", &num);
        if (num < 0)
            printf("Error: cannot find square root of a negative number.\n");
        else
            printf("Square root of %.2f = %.4f\n", num, squareRoot(num));
        break;

    case 6:
        printf("Exiting... Thank you!\n");
        break;

    default:
        printf("Invalid choice! Please select a number from 1 to 6.\n");
    }
} while (choice != 6);

return 0;
}

float circle(float radius)
{
    return PI * radius * radius;
}

float rectangle(float length, float width)
{
    return length * width;
}

float square(float side)
{
    return side * side;
}

double power(double x, double y)
{
    double result = 1.0;
    int i;
    for (i = 1; i <= y; i++)
        result = result * x;
    return result;
}

double squareRoot(double number)
{
    return sqrt(number);
}

void displaymenu(void)
{
    printf("\n===== CALCULATOR =====\n");
    printf("1. Area of Circle\n");
    printf("2. Area of Rectangle\n");
    printf("3. Area of Square\n");
}

```

```

    printf("4. Power\n");
    printf("5. Square Root\n");
    printf("6. Exit\n");
}

```

Output:

```

Lab5 Q1
Enter your choice: Enter a number: Square root of 16.00 = 4.0000

===== CALCULATOR =====
1. Area of Circle
2. Area of Rectangle
3. Area of Square
4. Power
5. Square Root
6. Exit
Enter your choice: Invalid choice! Please select a number from 1 to 6.

===== CALCULATOR =====
1. Area of Circle
2. Area of Rectangle
3. Area of Square
4. Power
5. Square Root
6. Exit
Enter your choice: Enter a number: Error: cannot find square root of a negative number.

===== CALCULATOR =====
1. Area of Circle
2. Area of Rectangle
3. Area of Square
4. Power
5. Square Root
6. Exit
Enter your choice: Exiting... Thank you!

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day5>mpt

```

Question 2: Number guessing game with random number generation

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int getUserGuess()
{
    int guess;
    printf("Enter your guess (1-100, 0 to exit): ");
    scanf("%d", &guess);
    return guess;
}

int checkGuess(int guess, int target)
{
    if (guess == target)
    {
        printf("Congratulations! You guessed the correct number: %d\n", target);
        return 1;
    }
    else if (guess > target)
        printf("Too high! Try again.\n");
    else
        printf("Too low! Try again.\n");
    return 0;
}

int main()
{
    int target, guess, correct = 0;

    srand(time(0));

```

```

target = (rand() % 100) + 1;

printf("===== NUMBER GUESSING GAME =====\n");
printf("I have chosen a number between 1 and 100.\n");
printf("Enter 0 at any time to exit the game.\n");
printf("=====\n\n");

while (correct == 0)
{
    guess = getUserGuess();

    if (guess == 0)
    {
        printf("\nYou chose to exit. The secret number was %d.\n", target);
        printf("Better luck next time!\n");
        break;
    }

    correct = checkGuess(guess, target);
}

printf("\n===== GAME OVER =====\n");
return 0;
}

```

Output:

```

q02guess
===== NUMBER GUESSING GAME =====
I have chosen a number between 1 and 100.
Enter 0 at any time to exit the game.
=====

Enter your guess (1-100, 0 to exit): Too low! Try again.
Enter your guess (1-100, 0 to exit): Too low! Try again.
Enter your guess (1-100, 0 to exit): Too high! Try again.
Enter your guess (1-100, 0 to exit): Too low! Try again.
Enter your guess (1-100, 0 to exit): Too high! Try again.
Enter your guess (1-100, 0 to exit): Too low! Try again.
Enter your guess (1-100, 0 to exit): Too high! Try again.
Enter your guess (1-100, 0 to exit): Too low! Try again.
Enter your guess (1-100, 0 to exit): Too high! Try again.
Enter your guess (1-100, 0 to exit): Too high! Try again.
Enter your guess (1-100, 0 to exit): Too low! Try again.
Enter your guess (1-100, 0 to exit):
You chose to exit. The secret number was 51.
Better luck next time!

===== GAME OVER =====

C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C\Day5>|

```

MCQ Questions: Answers

Q	Answer
1	C) void
2	A) int add(int, int);
3	B) Passing values of variables
4	C) return 0;
5	D) #include <math.h>

Q	Answer
6	A) srand(time(0))
7	C) rand() % 100 + 1
8	B) A function definition
9	D) Inside main() only
10	A) %f

Explain

1. Why should a C program use functions? Explain with an example.

Functions split a large program into small, reusable blocks. Each function does one job (input, calculation, output). This makes the code easier to read, debug, test and reuse. In the calculator program, the menu is a loop in main() while each calculation such as areaCircle() and squareRootNum() is a separate function. If a new operation is added later, only one new function and one new menu case are needed - nothing else changes.

2. What is the difference between a function declaration (prototype), a function definition, and a function call?

A declaration (prototype) only tells the compiler the name, return type and parameter types, e.g. float areaCircle(float radius);. A definition contains the actual body with the statements, e.g. float areaCircle(float radius) { return PI * radius * radius; }. A call transfers control to the function with actual arguments, e.g. areaCircle(radius);. A prototype may be repeated, but a definition may appear only once.

3. Explain call by value with the squareRootNum() example.

In C all arguments are passed by value: a copy of the value is made and given to the parameter. Inside squareRootNum(double num), num is a separate local variable holding a copy of the caller's value. Changing num inside the function never changes the original variable in main(). This protects the caller's data and is why the square root result must be returned instead of stored into the caller's variable directly.

4. Why is srand(time(0)) needed before calling rand()? What happens without it?

rand() produces a pseudo-random sequence that is the same every run unless the generator is seeded.

srand(time(0)) seeds the generator with the current time in seconds, so each run of the game starts at a different point of the sequence and picks a different target number. Without it, the target would be the same every time the program runs, making the game trivial.

5. In the guessing game, why does checkGuess() return 1 or 0 instead of only printing the hint?

Returning a flag lets main() know whether the game is over. checkGuess() compares the guess with the target and returns 1 when they match and 0 otherwise. The while loop in main() then runs until correct is 1. Printing alone would not tell the loop whether to continue, so the flag controls the game loop cleanly.